

Q. Write a program in python to create dataset having columns Student id, student name, age, height, weight, Semester, gender, city.

Perform following operations:

- (i) Shape of dataset
- (ii) Find name of columns
- (iii) Find the number of null values in each row.
- (iv) Find the unique names
- (v) Perform imputation to fill null values
- (vi) Plot line graph b/w height and weight, height and age, weight and age.
- (vii) Visualize outliers present in height columns using scatter plot.
- (viii) Show outliers present using box plot.
- (ix) Remove outliers using iqr method and z-score.

Ans:- import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

data = {

"Student ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

"Student_name": ["Amit", "Riya", "Sohan", "Neha", "Rahul",

"Priya", "Karan", "Sneha", "Vikas", "Rohit"],

"Age": [20, 21, 22, 20, 23, np.nan, 21, 23, np.nan, 21],

"Height": [165, 170, 172, 160, 180, 175, 168, 169, 167, 171],

"Weight": [55, 60, 65, 50, 75, 70, 62, 58, 90, 48, 59],

"Semester": [3, 4, 5, 3, 6, 5, 4, 4, 6, 3],

"Gender": ["M", "F", "M", "F", "M", "F", "M", "F", "M", "F"],

"City": ["Delhi", "Mum", "Pune", "Utt", "Pune", "Delhi",
"Mum", "Pun", "Delhi", "Upp"]

}

df = pd.DataFrame(data)

print("Dataset:\n", df)

print("\n Shape of DataSet:", df.shape)

print("\n Column Names")

df.columns.tolist()

print("\n Null values in Each Row:")

print(df.isnull().sum(axis=1))

```
print("\n Unique Student names")
print(df["Student_names"].unique())
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Weight"] = df["Weight"].fillna(df["Weight"].mean())
```

```
plt.plot(df["Height"], df["Weight"])
plt.xlabel("Height")
plt.ylabel("Weight")
plt.show()
```

```
plt.plot(df["Height"], df["Age"])
plt.xlabel("Height")
plt.ylabel("Age")
plt.show()
```

```
plt.plot(df["Weight"], df["Age"])
plt.xlabel("Weight")
plt.ylabel("Age")
plt.show()
```

```
plt.scatter(df["Height"], df["Age"])
plt.xlabel("Height")
plt.ylabel("Age")
plt.show()
```

```
plt.boxplot(df["Height"])
plt.title("Box Plot of Height")
plt.show()
```

```
q1 = df["Height"].quantile(0.25)
q3 = df["Height"].quantile(0.75)
IQR = q3 - q1
df_iqr = df[(df["Height"] >= q1 - 1.5 * IQR) and (df["Height"] <= q3 + 1.5 * IQR)]
print("\n Dataset after removing outliers")
print(df_iqr)
```

```
z_score = (df["Height"] - df["Height"].mean()) / df["Height"].std()
df_zscore = df[abs(z_score) <= 3]
print("\n Dataset after removing outliers using z-score")
print(df_zscore)
```

Handwritten signature and date:
 27/10/26

Q2(i) Develop a python program to create a dataset having height, weight, gender as columns using numpy functions `randint`, `random`, `uniform` and `choice` function (100 entries)

(ii) Calculate BMI using database operations. Save this in CSV.

(iii) Import the CSV file.

(iv) Find Shape and Size of dataframe.

(v) Find the count of null values in gender column.

(vi) Perform Imputation to remove null values from gender.

(vii) Find the structural and statistical information of dataset.

(viii) Create line graph btw height and weight. and analyse if it is a suitable graph or not. If not plot suitable graph.

(ix) Create histogram on gender column.

(x) Perform aggregate function on height, weight and BMI columns.

(xi) Find entries where BMI < 20.

Soln

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(1)
height = np.random.randint(150, 190, 100)
weight = np.random.uniform(45, 95, 100)
gender = np.random.choice(['Male', 'Female', None], 100, p=[0.45, 0.45, 0.1])
df = pd.DataFrame({
    'Height': height,
    'Weight': weight,
    'Gender': gender})
print(df.head())

df['BMI'] = df['Weight'] / ((df['Height']/100)**2)
print(df.head())

df.to_csv('bmi_data.csv', index=False)

data = pd.read_csv('bmi_data.csv')

print("Shape:", df.data.shape)
print("Size:", data.size)

print("Null values in Gender:", data['Gender'].isnull().sum())
data['Gender'].fillna(data['Gender'].mode()[0], inplace=True)
```

93

```

print(data.info())
print(data.describe())

plt.plot(data['Height'], data['Weight'])
plt.show()
plt.scatter(data['Height'], data['Weight'])
plt.xlabel('Height (cm)')
plt.ylabel('Weight (Kg)')
plt.title('Height vs Weight')
plt.show()

data['gender'].value_counts().plot(kind='bar')
plt.title('Gender Distribution')
plt.show()

print(data[['Height', 'Weight', 'BMI']].agg(['min', 'max', 'mean']))
print(data[data['BMI'] < 20])

```

Q Perform EDA on medical_record.csv and patient_detail.csv.

Ans: import pandas as pd
import matplotlib.pyplot as plt

```

medical = pd.read_csv("medical_record.csv")
patients = pd.read_csv("patient_details.csv")
patients["gender"] = patients["gender"].replace({
    "F": "Female", "M": "Male"})
patients["city"] = patients["city"].str.title()
patients["name"] = patients["name"].fillna("Unknown")
patients["height_cm"] = pd.to_numeric(patients["height_cm"], errors="coerce")
patients["height_cm"].fillna(patients["height_cm"].mean(), inplace=True)
patients["age"].fillna(patients["age"].mean(), inplace=True)

medical["bp"] = medical["bp"].replace("high", "140/90")
medical["sugar_level"].fillna(medical["sugar_level"].mean(), inplace=True)

merge_df = pd.merge(patients, medical, on="patient_id", how="inner")
merge_df.drop_duplicates(inplace=True)

```

```
print("In Missing Values")  
print("merge_df.isnull().sum()")  
print(df.info())  
print(merge_df)
```

27/10/26

Depth First Search

```
def dfs(start, goal, graph, visited=None):
    if visited is None:
        visited = Set()
    print(start, end=" ")
    if start == goal:
        return True
    visited.add(start)
    for neighbour in graph[start]:
        if neighbour not in visited:
            if dfs(neighbour, goal, graph, visited):
                return True
    return False

graph = {'A': ['B', 'C', 'D'],
        'B': ['C', 'D'],
        'C': ['A', 'B', 'D'],
        'D': ['A', 'C', 'B']}

Ar dfs('A', 'D', graph)
```

Q4 Design Develop a program to implement depth first search for graph traversal with following specification. Construct a static graph of 6 nodes and 10 edges where graph is stored in dictionary. $A \rightarrow B, A \rightarrow C, B \rightarrow D, B \rightarrow E, C \rightarrow D, C \rightarrow F, C \rightarrow E, D \rightarrow F, E \rightarrow F, B \rightarrow F$. Create another graph where number of nodes and no of edges are provided by user at runtime. Accept initial and goal node from user. Display path from initial to goal node.

Ans:-

```
def dfs(start, goal, graph, visited=None):
    if visited is None:
        visited = Set()
    print(start, end=" ")
    if start == goal:
        return True
    visited.add(start)
    for neighbour in graph[start]:
```

if neighbour not in visited:

if dfs(neighbour, goal, graph, visited):

return true

return False

```
graph = {'A': ['B', 'C'],  
        'B': ['D', 'E', 'F'],  
        'C': ['D', 'E', 'F'],  
        'D': ['B', 'C', 'F'],  
        'E': ['B', 'C', 'F'],  
        'F': ['B', 'C', 'D', 'E'],  
        }
```

dfs('A', 'E', graph)

output:- ABDCE

(ii) nodes = int(input("enter how many nodes"))

l = []

graph = {}

for i in range(nodes):

node = input(f"enter {i+1} node:")

neighbour = int(input(f"enter total neighbour of {node}"))

for j in range(neighbour):

x = input(f"enter the {j+1} neighbour of node {node}")

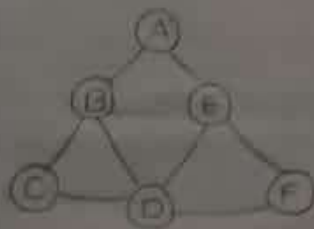
l.append(x)

graph.update({node: l})

start = input("Enter the start node:")

goal = input("Enter the goal node:")

dfs(start, goal, graph)



output → ABCDE

Q5 Design and develop a program to implement BFS with following Specifications

- (i) Construct a Static graph consisting of 6 nodes and 10 edges where the graph is stored in dictionary
 $A \rightarrow B, A \rightarrow C, B \rightarrow D, B \rightarrow E, C \rightarrow D, C \rightarrow F, C \rightarrow E, D \rightarrow F, E \rightarrow F, B \rightarrow F$
- (ii) Create another graph where number of nodes and number of edges are provided by user at runtime.
- (iii) Accept the initial and goal node from user. Display path for initial to goal node.

Ans $\rightarrow$ def bfs (st, goal, graph):

 queue = [st]

 path = []

 vis = set()

 vis.add(st)

 while queue:

 node = queue.pop(0)

 path.append(node)

 if node == goal:

 return path

 for neighbour in graph[node]:

 if neighbour not in vis:

 vis.add(neighbour)

 queue.append(neighbour)

 return path

graph = {'A': ['B', 'C'],
 'B': ['D', 'E', 'F'],
 'C': ['D', 'E', 'F'],
 'D': ['B', 'C', 'F'],
 'E': ['B', 'C', 'F'],
 'F': ['B', 'C', 'D', 'E']}

 bfs('A', 'E', graph)

output $\rightarrow$ ABOCE

(ii) nodes = int(input("How many nodes"))

l = []

graph = {}

for i in range(nodes):

 nodes = input(f"Enter {i+1} node: ")

 neighbour = int(input(f"Enter total neighbour of node {nodes}"))

 for j in range(neighbour):

```

x = input ("Enter the {j+1} neighbour of node {node}")
l.append(x)
graph.update ({node: l})
start = input ("Enter the start node")
goal = input ("Enter the goal node")
bfs (start, goal, graph)

```



output $\rightarrow$ ABCDE

(G) # Algorithm for A*

```

import networkx as nx
graph = nx.Graph()

```

while True:

```

choice = int(input ("Want to insert nodes and edges press? 1 for yes
2 for no"))

```

if choice == 1:

```

node1 = input ("Enter node 1").strip().upper()

```

```

node2 = input ("Enter node 2").strip().upper()

```

```

weight = int(input ("Enter weight btw node1 and node2"))

```

```

graph.add_edge (node1, node2, weight=weight)

```

else:

```

break

```

```

for node, neighbour in graph.adj.items():

```

```

    print (node, neighbour)

```

```

start = input ("Enter start node").strip().upper()

```

```

goal = input ("Enter goal node").strip().upper()

```

```

heuristic_values = {}

```

```

heuristic_values[goal] = 0

```

```

for node in graph.nodes():

```

```

    if node != goal:

```

```

        heuristic_val = int(input ("Enter the heuristic value from
node {node} to goal node {goal}"))

```

```

        heuristic_values[node] = heuristic_val

```

```

heuristic_values

```

```

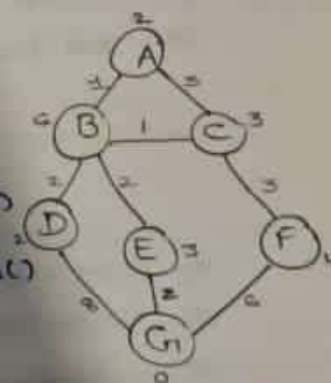
def get_heuristic (u,v):

```

```

    return heuristic_values[u]

```



Total cost $\rightarrow$ 5

Path $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ F

```

path = nx.astar_path(graph, start, goal, heuristic = get_heuristic, weight = 'weight')
"→".join(path)
total_cost = nx.astar_path_length(graph, start, goal, heuristic = get_heuristic, weight = 'weight')
total_cost

```

Write a python program to create dataset having cgpa in range 1-5, height in cmeter, marks in range 1 to 100, fees in range 10-50 lakhs.

- Perform min max normalization from scratch
- Implement standardization using standard score from scratch.
- Perform min-max normalization using library
- Perform standardisation using library.

```

Ans
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(1)
cgpa = np.random.randint(1, 6, 10)
height = np.random.randint(150, 191, 10)
marks = np.random.randint(1, 101, 10)
fees = np.random.randint(1000000, 5000000, 10)
df = pd.DataFrame({'CGPA': cgpa,
                    'Height': height,
                    'Marks': marks,
                    'Fees': fees})

```

(i) $df_copy = df$
 $df_normalized = (df_copy - df_copy.min()) / (df_copy.max() - df_copy.min())$
 $df_normalized$

(ii) $Standardized_df = (df_copy - df_copy.mean()) / df_copy.std()$
 $Standardized_df$

(iii) from sklearn.preprocessing import MinMaxScaler
 $mms = MinMaxScaler()$
 $x = mms.fit_transform(df)$
 $print(x)$

```
(iv) from sklearn.preprocessing import StandardScaler
mms = StandardScaler()
x1 = mms.fit_transform(df)
print(x1)
```

Q8 Develop a python program to perform normalization and standardisation on a given health care dataset using minmax scaler and standard scaler. The program should:

- load given dataset from csv file
- Apply minmax normalisation
- Apply standard scaler to given dataframe.
- Plot appropriate to visualise normalized data obtained from both methods.

Ans:- import pandas as pd

(i) df = pd.read_csv('healthcare.csv')

(ii) df.isnull().sum()

df_copy = df

df_normalized = (df_copy - df_copy.min()) / (df_copy.max() - df_copy.min())

df_normalized

(iii) df_copy1 = df

standardized_df = (df_copy1 - df_copy1.mean()) / df_copy1.std()

standardized_df

from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler()

x = mms.fit_transform(df)

print(x)

from sklearn.preprocessing import StandardScaler

mms = StandardScaler()

x1 = mms.fit_transform(df)

print(x1)

Amal Kumar Kohli
29/3/26

(iv) import matplotlib.pyplot as plt

plt.hist(df_normalized)

plt.show()

plt.boxplot(standardized_df)

plt.grid(True)

plt.show()

Q9 Build a Python Program to implement Forward Feature Selection and Backward feature Elimination techniques using a given dataset. The program should perform the following:

- (i) Load the dataset into dataframe
- (ii) Apply Forward Feature Selection using a linear regression model.
- (iii) Apply Backward Feature Elimination using same machine learning model and analyze the result.
- (iv) Compare the feature sets selected by Forward Feature Selection and Backward Feature Elimination.

Ans `import pandas as pd`

`from sklearn.linear_model import LinearRegression`

`from sklearn.feature_selection import SequentialFeatureSelector`

`data = pd.read_csv("feature-selection.csv")`

`x = data.iloc[:, :-1]`

`y = data.iloc[:, -1]`

`model = LinearRegression()`

`fs = SequentialFeatureSelector(model, n_feature_to_select=3,`
`fs.fit(x, y) direction = 'forward')`

`Selected_feature = x.columns[fs.get_support()]`

`be = SequentialFeatureSelector(model, n_feature_to_select`
`= 3, direction = 'backward')`

`be.fit(x, y)`

`Selected_features = x.columns[be.get_support()]`

`fs.get_support()`

Q10 You are given a dataset (salary-prediction.csv) containing years-of-experience as input feature and Salary as target. Implement a program to perform Simple Linear Regression and analyze relationship btw input features and target Variable and perform the following:

- (i) Read the dataset into a dataframe and train a simple linear regression model after splitting the dataset into train and test data in 70:30.
- (ii) Determine Slope and Intercept of the best fit line.
- (iii) Plot the regression line along with actual data point.
- (iv) Use the trained model to predict Salary of new data points.
- (v) Evaluate the performance of the model using appropriate regression metrics.

```
Ans import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

data = pd.read_csv('Salary-prediction.csv')
df = pd.DataFrame(data)
x = df[['Experience']]
y = df['Salary']

x_train, x_test, y_train, y_test = train_test_split(x, y,
                                                    test_size=0.3,
                                                    random_state=42)

model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
```

```
Slope = model.coef_ [0]  
intercept = model.intercept_  
Slope, intercept.
```

```
import matplotlib.pyplot as plt  
plt.scatter(x, y, color="blue", label="Actual Data")  
plt.plot(x, model.predict(x), color="red")  
plt.xlabel("Experience")  
plt.ylabel("Salary")  
plt.title("Linear Regression Best Fit Line")  
plt.legend()
```

```
data_point = int(input("Enter experience"))  
print("Expected Salary is:", model.predict(data_point))
```

```
print("Mean absolute error is")  
mean_absolute_error(y_test, y_pred)  
print("Mean squared error is")  
mean_squared_error(y_test, y_pred)  
print("R2 score is")  
r2_score(y_test, y_pred)
```

Q1. Consider a data set (salary-prediction.csv) containing multiple input features related to employee performance and a target variable Salary. Implement a program to perform Multiple Linear Regression and analyze the relationship between multiple input feature and target variable.

- (i) Read the given dataset into dataframe
- (ii) Split the data set into training and testing set in ratios 70:30, 60:40, 80:20
- (iii) Train a multiple linear Regression model using all input features to predict target variable Salary.
- (iv) Determine coefficient of each input feature and intercept of best-fit regression model.

```

Ans import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score

cgpa = [10, 9, 9.7, 7.8, 6.8, 9.2, 7.8, 7.8, 6.8],
package = [23, 20, 21, 19, 17, 18, 26, 17, 15, 18]
iq = [100, 120, 140, 123, 145, 122, 170, 120, 100, 145]

df = pd.DataFrame ( {"CGPA": cgpa,
                     "IQ": iq,
                     "Salary": package})

```

```

x = df.iloc[:, 0:-1]
y = df.iloc[:, -1]
x_train1, x_test1, y_train1, y_test1 = train_test_split(x, y,
                                                         test_size = 0.3,
                                                         random_state = 42)

```

```

x_train2, x_test2, y_train2, y_test2 = train_test_split(x, y,
                                                         test_size = 0.4,
                                                         random_state = 42)

```

```

x_train3, x_test3, y_train3, y_test3 = train_test_split
(x, y, test_size = 0.2,
random_state = 42)

```

```

modell = LinearRegression()
modell.fit(x_train1, y_train1)
y_pred1 = modell.predict(x_test1)
y_pred1

```

```

i = 1
for i in modell.coef_:
    print(f"coef of {i} feature is {i}")
print("Intercept is", modell.intercept_)

```

```

model2 = LinearRegression()
model2.fit(x_train2, y_train2)
y_pred2 = model2.predict(x_test2)

```

```

j = 1
for i in range(model2.coef-):
    print(f"coef of {i}th feature is {i}")
    j = j + 1
print(model2.intercept-)

```

```

model3 = LinearRegression()
model3.fit(x_train3, y_train3)
y_pred3 = model3.predict(x_test3)

```

```

j = 1
for i in range(model3.coef-):
    print(f"coef of {i}th feature is {i}")
    j = j + 1
print(model3.intercept-)

```

Q12. You are working on a dataset (disea_risk.csv) containing multiple input features related to patient health parameters and binary target variable Disease_Risk. Implement a program to perform Logistic Regression for binary classification and analyze the relationship b/w the input features and target variable. Perform the following tasks

- (i) Read the given dataset into dataframe.
- (ii) Split the dataset into training and testing set in ratio 70:30
- (iii) Train Logistic Regression model using the input features to predict the binary target variable.
- (iv) Determine model coef and intercept.
- (v) Evaluate the performance of model using accuracy and show confusion matrix
- (vi) Use the trained model to predict the class label for new datapoints.

```
Ans import numpy as np
import pandas as pd
import sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
data = pd.read_csv('dis2a_risk.csv')
df = pd.DataFrame(data)
```

```
x = df.iloc[:, :-1]
y = df.iloc[:, -1]
x_train, x_test, y_train, y_test = train_test_split(x, y,
                                                    test_size=0.3,
                                                    random_state=42,
                                                    stratify=y)
```

```
model = LogisticRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
```

```
slope = model.coef_
intercept = model.intercept_
print(slope)
print(intercept)
```

```
print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

```
# Assuming there are two features:-
x_point = int(input("Enter first feature"))
y_point = int(input("Enter second feature"))
new_data = [[x, y]]
print(model.predict(new_data))
```